CS24301 — THEORY (3.0 CR) | MODULE III NOTES | BIT MESRA | NEP SCHEME

Module 3: Semantic Analysis & Type Systems

SDD, SDTS, Synthesized/Inherited, Type Checking & Symbol Tables

Module III: Semantic Analysis & Type Checking — Complete Syllabus Topics

1. Syntax-Directed Definitions (SDD) & Semantic Rules
2. Synthesized Attributes vs. Inherited Attributes
3. S-Attributed Definitions & Bottom-Up Evaluation
4. L-Attributed Definitions & Top-Down Evaluation Orders
5. Dependency Graphs & Topological Sorting for Attribute Evaluation
6. Syntax-Directed Translation Schemes (SDTS) & Action Placement
7. Static Type Checking & Type Systems
8. Type Expressions, Type Equivalence (Structural vs Name)
9. Type Conversions (Implicit Coercion vs Explicit Casting)
10. Symbol Table Organizations (Hash Tables with Scope Chaining)

Topic 1 & 2: Syntax-Directed Definitions (SDD) & Attributes

A **Syntax-Directed Definition (SDD)** is a context-free grammar together with attributes and semantic rules. Attributes are associated with grammar symbols, and rules are associated with productions.

Attribute Class	Definition & Value Flow Direction	Key Evaluation Properties
1. Synthesized Attribute	Computed from the attribute values of the node's children in the parse tree (or lexical values). $$A.s = f(Y_1.a_1, Y_2.a_2, \ldots, Y_k.a_k)$$	Flows strictly **Bottom-Up**; naturally evaluated during LR parsing.
2. Inherited Attribute	Computed from the attribute values of the node's parent and/or siblings. $$Y_j.i = f(A.a, Y_1.a_1, \ldots, Y_{j-1}.a_{j-1})$$	Flows **Top-Down** and **Left-to-Right**; passes context (e.g., data type declarations) downward.

Topic 3 & 4: S-Attributed vs. L-Attributed SDD

Structural Classifications

- **S-Attributed SDD:** An SDD that uses **only synthesized attributes**. Can be evaluated during bottom-up parsing (e.g., LR parser) by maintaining an attribute stack in parallel with the parse stack.
- **L-Attributed SDD:** An SDD where every attribute is either synthesized, or inherited with the restriction that for production $A \to X_1 X_2 \ldots X_n$, an inherited attribute of X_j depends only on:

1. Attributes of the parent A.
2. Attributes of symbols to the left of X_j ($X_1, X_2, \ldots, X_{j-1}$).

Every S-Attributed SDD is strictly an L-Attributed SDD ($S\text{-Attributed} \subset L\text{-Attributed}$).

🏛️ Step-by-Step Solved Problem: Desktop Calculator S-Attributed SDD

Production	Semantic Rule
$L \to E\ \mathbf{n}$	$\text{print}(E.\text{val})$
$E \to E_1 + T$	$E.\text{val} = E_1.\text{val} + T.\text{val}$
$E \to T$	$E.\text{val} = T.\text{val}$
$T \to T_1 * F$	$T.\text{val} = T_1.\text{val} * F.\text{val}$
$T \to F$	$T.\text{val} = F.\text{val}$
$F \to (E)$	$F.\text{val} = E.\text{val}$
$F \to \mathbf{digit}$	$F.\text{val} = \mathbf{digit}.\text{lexval}$

Evaluates $(3+4)*5\mathbf{n} \implies \mathbf{35}$ bottom-up in a single pass.

Topic 7 & 8: Type Checking & Type Systems

A **Type Checker** verifies that the type of a construct matches that expected by its context (e.g., the modulo operator `%` requires integer operands in C).

1. Structural vs. Name Equivalence:

Name Equivalence: Two types are equivalent if and only if they share the exact same type name identifier (Strict, fast to check).
Structural Equivalence: Two types are equivalent if they have identical internal structures (e.g., record fields with identical names and types in the same order).

2. Type Conversions:

Implicit Coercion: Automatically performed by compiler (e.g., widening an `int` to `float` during addition `2 + 3.5`).
Explicit Casting: Programmed explicitly by the developer (e.g., `(int) x`).

Topic 10: Symbol Table Organization

The **Symbol Table** is a critical data structure containing record entries for every user-defined identifier in the source program, storing name, type, memory offset, scope level, and parameter signatures.

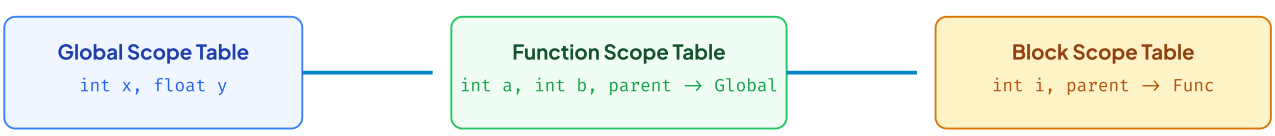

Figure 3.1: Hierarchical Block-Structured Scope Chaining

🏛️ Top BIT Mesra Exam Questions & Answers (Module III)

Q1. Differentiate between S-Attributed and L-Attributed SDD with syntax-directed rules. (8 Marks)

1. **S-Attributed SDD:** Uses only synthesized attributes. Can be evaluated during bottom-up parsing using an attribute stack without creating an explicit parse tree.
2. **L-Attributed SDD:** Allows both synthesized and inherited attributes, but inherited attributes of symbol X_j in $A \to X_1 X_2 \dots X_n$ are restricted to depend only on attributes of parent A and left siblings $X_1, \dots, X_{j-1}$. Can be evaluated in a single depth-first, left-to-right parse tree traversal.

Q2. How is dynamic block scoping implemented in a symbol table? (6 Marks)

Block scoping is implemented using a **Stack of Hash Tables** or a chained tree of symbol tables. When entering a new block `{`, a new empty hash table is created and pushed onto the scope stack, with a pointer pointing to its enclosing parent scope table. Symbol lookups search from the current top-of-stack table upwards along parent pointers. When exiting a block `}`, its table is popped and deallocated.